Machine Translated by Google

RFI
RETE FERROVIARIA ITALIANA
GRUPPO FERROVIE DELLO STATO ITALIANE

Interoperability Specifications for Communication with Systems field

Machine Translated by Google

RFI RETE FERROVIARIA ITALIANA
GRUPPO FERROVIE DELLO STATO ITALIANE

SUMMARY 1.

General concepts 3

1.1. Introduction 3

1.2. Scope, context and objectives 3

1.2.1. Scope..... 3

1.2.2. Context and Assumptions..... 4

1.2.3. Objectives of the Specification 6

2. Communication Modes Field Systems Unit – IoT Platform..... 6

2.1 Authentication of Management and Edge Units..... 6

2.1.1 Authentication with JWT Tokens 6

2.2 Specification Summary 7

2.3 Protocols 8

2.3.1 MQTTS Protocol 8

2.3.2 SFTP Protocol..... 8

2.3.3 HTTPS Protocol 8

3. Alert Management and Data Flow..... 9

3.1 Alarm Management..... 9

3.1.1 Periodic Alarm Realignment 11

3.2. Fault Management 12

3.2.1 Periodic Fault Realignment..... 15

3.3 Measure Management..... 15

3.3.1 PULL Mode 17

3.4 Snapshot Management..... 18

3.4.1 Mass image sending..... 19

3.5 Topics to use..... 21

4. Device Management 22

4.1 Management of the Personal Data 22

4.2 Command Management..... 23

4.3 Topics to use..... 25

5. Remote access to field equipment by RFI..... 26

6. Remote access to field devices by Suppliers..... 26

Machine Translated by Google

1. General concepts

1.1. Introduction

This document aims to define the management and interaction processes between local OT networks (monitoring systems installed in the field) and the IoT Central Platform, ensuring the completeness and accuracy of the information collected in the field.

The standards covered impact the following operating contexts:

- **Device Management:** management of the **operational configurations of the devices** that they make up the physical monitoring infrastructure;
- **Data Flow Management:** processes for **sharing and processing** recorded and/or processed locally by field systems towards the IoT Central Platform;
- **Alert Management:** notification of **faults** or **alarms** to the IoT Central Platform, in the event of threshold exceedance, malfunctions or changes in network status.

Please note that, during the implementation phase, the level of coverage of the features expected for each operating context may vary based on the type of components chosen for the specific system; therefore, for example, on less complex networks it may be possible to have a limited set of features.

The objectives to be achieved are the following:

- Ensure, regardless of the nature of the installed hardware, interoperability with the IoT platform;
- Make measurement information from the field (e.g. Alerts) available continuously and promptly (e.g. for real-time Alerts);
- Enable the analysis and processing of extremely high volumes of data;
- Enable integrated device management of ecosystem systems and devices;
- Ensure traceability of data sent from OT devices.

1.2. Scope, context and objectives

1.2.1. Scope

RFI's real estate and infrastructure assets are varied and characterized by different construction techniques and materials used over time.

For this reason, it is particularly useful to have a continuous and efficient IoT system over time, which allows you to:

- Monitor and verify the conditions of the infrastructure components; • Monitor and identify external causes that may cause any failures or malfunctions; •
- Support the infrastructure inspection and maintenance process.

Typically, an IoT platform consists of three levels:

- EDGE TIER: This is the layer where measurements are collected from the OT network environment, sampled, and transferred according to appropriate logic to the upper layer. This layer includes both the OT network and the Edge Gateways for translating, filtering, and sending data to the IoT platform.

Machine Translated by Google

- PLATFORM TIER: consists of the central IoT platform that performs global operations in collaboration with the Edge Tier such as device management, stream processing, advanced analytics and workload orchestration
- ENTERPRISE TIER: This is the layer where the IoT platform integrates with corporate IT applications and processes, such as ERP. Corporate applications can benefit from data-driven insights from the IoT platform.

(Gartner reference "Architect IoT Using the Gartner Reference Model")

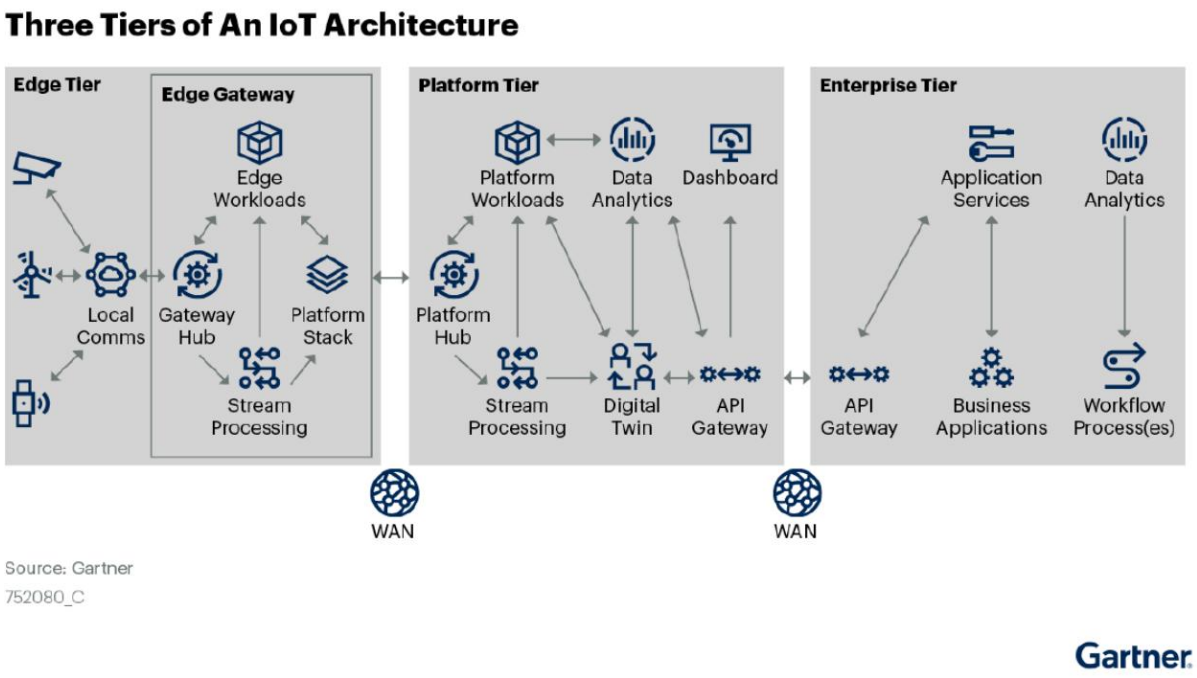

Figure 1 – Three Tiers of AN IoT Architecture.

Using an IoT platform that operates not only locally but across the entire supply chain will enable improved inspections and maintenance.

For example, by using historical data from field systems, in addition to that recorded by instruments during instrumental visits, it will be possible to integrate the existing information regarding the state of conservation of the works to define representative indicators, supporting RFI users in their decision-making.

The bridge monitoring model, which will be described below, supported by appropriate technological solutions, will also provide information for real-time monitoring of the status of the work and for event/ fault management.

1.2.2. Context And Hiring

The reference context in question includes defining the integration solution between RFI's IoT Platform and monitoring systems. The following is the logical architecture representing the bridge monitoring systems.

Machine Translated by Google

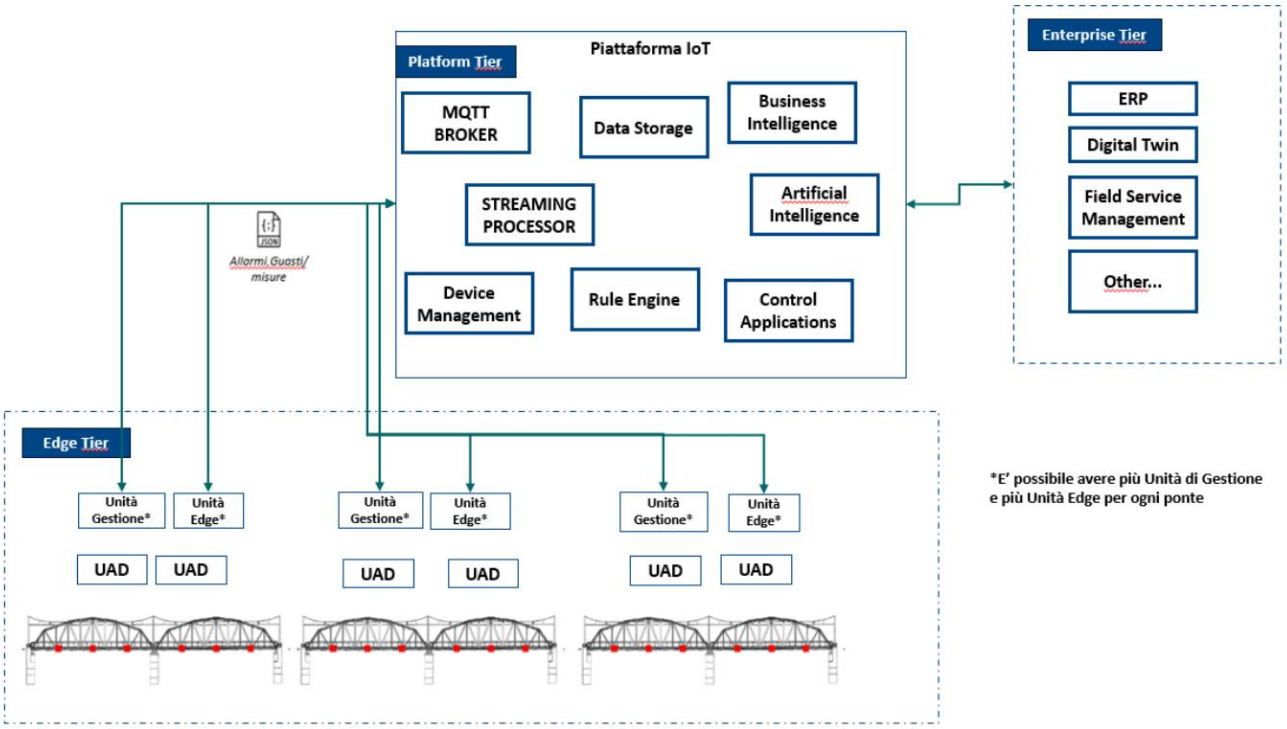

Figure 2 - Logical Architecture

As shown in Figure 2 – Logical Architecture, the data detected by the sensors installed on the bridges are collected by the Data Acquisition Units (DAU) to which they are directly connected.

The field architecture includes at least one UAD for each instrumented span, capable of collecting information from the various sensors installed on the particular monitored span.

Each UAD sends data to a Management Unit. Each Management Unit sends data to one or more Edge units, consisting of an industrial PC, positioned inside an outdoor cabinet near the instrumented structure. Depending on the number of spans, each bridge can have multiple Management Units and Edge Units.

The Management Unit manages and configures the connected UADs (see § 5.7 of the Technical Specification for the tender), while the Edge Unit is the module in which the algorithms defined by RFI are executed. In parallel, both units can interface with the IoT platform.

The developments on the Edge Unit and the Management Unit will be entirely borne by RFI and the respective suppliers according to the tender specifications and to date no product installations by FS Technology are foreseen on these components.

The IoT platform provides advanced capabilities including data streaming, historicization, and analytics.

The data acquired in the IoT platform will be used for various purposes of the maintenance process:

- Alarm/fault management;
- Maintenance process optimization;
- Acquisition and logging of measurements for timely analysis of the state of the artwork.

Machine Translated by Google

The logical architecture shown (Fig. 2) provides for a two-way data exchange between the field units (Management and Edge) and the IoT Platform. In fact, there will also be a dedicated *flow* to the Management Units or Edge Units:

- to the transmission of status changes made at central level of faults and alarms respectively;
- to the transmission of commands for the management of sensor devices.

1.2.3. Objectives of the Specification

The objective of the specification is to define communication methods for integrating the various types and families of data between the Management and Edge Units and RFI's IoT platform. The objectives of the specification are divided into three macro-functional contexts:

- Alert Management - involves the detection, filtering and notification of alarm events or failures towards the IoT Platform;
- Data Flow Management - concerns the processes and protocols for acquiring and transferring field-measured quantities to the centralized data storage, processing, and visualization systems of RFI's IoT platform. This also includes the data produced as output by the processing algorithms residing at the Edge (e.g., structure frequencies, performance indicators, etc.).

- Device Management - concerns the management of the status and operational configurations of the devices that make up the physical monitoring infrastructure such as sensors, acquisition systems and field processors (e.g. Management Units, Edge Units, UADs, Dataloggers, Local Data Storage Units, etc.)

The specifications in this document will be illustrative in nature, with a good level of detail, but they will not be completely exhaustive as they will be detailed during the implementation phase. Therefore, any changes are subject to change. Such changes will be communicated prior to the start of field systems development and will not affect the nature and *effort* expected for the development of the field systems contractors.

2. Communication Modes Field Systems Unit – IoT Platform

2.1 Authentication of Management and Edge Units

Authentication will involve the use of a TLS digital certificate (X.509) by the Management Unit/Edge Unit to communicate information to the IoT platform.

The following differentiation is expected depending on the protocol used:

Protocol	Authentication
MQTTS User	+ password
SFTP	Username + password
HTTPS	JWT Token

Additionally, an Access Control List is provided to manage data sending authorization for a particular sensor/UAD/ Management Unit/Edge Unit/etc.

2.1.1 Authentication with JWT Token

In order for the Management Unit/Edge Unit and the IoT platform to exchange data, in the case of HTTPS communication, it is necessary to include the JWT authentication token in the HTTP header.

Machine Translated by Google

The process of creating the JWT token is as follows:

- Token Request: Before making an API call, the device requires a JWT token to the Identity Provider (IdP)
- If the authentication is valid, the authentication server returns a JWT token to the caller
- Token Usage for REST API Calls: Once obtained, the JWT token is included in the HTTP Authorization header of each request. Specifically, the token is passed in the HTTP Authorization header of each request: e.g., GET https://{URL}/api/devices/status Authorization: Bearer eyJhbGciOiJIUzI1NiIs....

2.2 Specification Summary

The following table provides, based on the type of flows, a summary of the protocols and data formats used in the messages exchanged between the Management Unit/Edge Unit and the IoT Platform.

During the implementation phase, detailed implementation cases will be evaluated.

The table below defines the Quality of Service levels of the MQTT protocol, but this parameter must be configurable and therefore must be modifiable during the implementation phase without any implementation impact.

Flow	Case Studies of Use	Protocol	Data Format
Alert Management - Alarm Management	Near-Real Time Alarm Sending	MQTTS QoS=1	json
Alert Management - Fault Management	Near-Real Time Fault Reporting	MQTTS QoS=1	json
Data Flow Management - Measurements and Results Algorithms	Send measurement/result at reduced frequency and small payload size (KB order)	MQTTS QoS=0	json
	Massive sending of measurement/ result with medium and large payloads	SFTP; SFTP+HTTPS	CSV + ZIP
Data Flow Management - Snapshot	Sending small images in Base64 and at a reduced frequency	MQTTS QoS=0	json
	Bulk sending of medium and large sized images	SFTP; SFTP+HTTPS	ZIP Images
Device Management	Sending commands	MQTT QoS=1	json

The document defines the data exchange methods, but the definition of field domains (i.e., master data) and the related valuation logic are not covered by these specifications and will be defined during the implementation phase between RFI and the field systems supplier. Furthermore, these master data must be communicated to FS Technology accordingly.

The frequency of data transmission to the IoT Platform will be determined on a case-by-case basis between RFI and the supplier. *FS Technology will need to be involved to assess the impact on the IoT Platform.*

Machine Translated by Google

2.3 Protocols

2.3.1 MQTTS Protocol

The MQTTS protocol supports real-time and/or near-real-time communication between the Management Unit/Edge Unit and the IoT platform in both directions. It is a lightweight messaging protocol designed for applications requiring low data impact and/or limited bandwidth. Adopting this protocol therefore aims to standardize communication with various vendors' field systems, as well as ensure data stability and persistence.

In particular, the MQTTS protocol is a standard ISO protocol capable of guaranteeing:

- Homogeneous standard protocol;
- Data stability and persistence; •
  Continuous and unsolicited mode (Push); •
  Bidirectional asynchronous transmission (from the field to the platform and vice versa); • Low impact on the network and processing resources.

Details of the selected protocol are below:

- MQTT 5.0 (release date 2019) defines a TCP/IP data transmission protocol based on a publish/subscribe model that operates through a dedicated message broker located in the IoT Platform. If a later version is used, it must be backwards compatible with version 5.0.

- TLS (Transport Layer Security) is used to encrypt communication between the client and the MQTT broker.
- The port used by the MQTT protocol must be configurable and will be specified during the implementation phase.

- The Quality of Service can be of type 1 or 0 depending on the flow, and must be a configurable parameter and therefore must be modifiable during the implementation phase without any implementation impact.

2.3.2 SFTP Protocol

The use of the SFTP protocol is foreseen in the case of batch transmission of data from the Unit of Edge Management/Unit to IoT Platform

SFTP (Secure File Transfer Protocol) is a network protocol designed for secure file transfer over an encrypted connection. Based on the SSH (Secure Shell) protocol, SFTP provides strong authentication and data protection during transmission, preventing interception and tampering.

SFTP is widely used to ensure the integrity and confidentiality of sensitive information, such as operational data, reducing the risk of unauthorized access and man-in-the-middle attacks.

2.3.3 HTTPS Protocol

This protocol is expected to be used in case of a request via API by the IoT Platform to re-forward data in the event of a fault or to send implementation commands to the field component (Device Management). The HTTPS (Hypertext Transfer Protocol Secure) protocol is a communication protocol used for the secure transmission of data over the Internet. It is based on the HTTP (Hypertext Transfer Protocol), but adds a layer of encryption and authentication to ensure the

Machine Translated by Google

Privacy and security of data exchanged between the client and the server. HTTPS is widely used to protect the confidentiality of sensitive information.

3. Alert Management and Data Flow

The information transmitted by the monitoring system to the platform has been classified into the following **data catalogue** (alarms, measurements, faults):

1) Alarms: These indicate the abnormality of a specific parameter and provide a logical True/False status, complemented by indications of the alarm's increasing intensity. These are further classified into:

- **Pre-alert (Events):** Refers to the pre-alert level and with increasing intensity.
- **Alert (Alarm):** Relating to the abnormal state of a specific parameter, defined by a logical True/False state and completed by indications for the alarm level, with increasing intensity.

The distinction between messages containing event data and those related to alarms is managed through the "AlarmSeverityLevel" field in the JSON, described below. The classification will be established by RFI when defining the master data.

2) **Faults** (or failures): Information regarding the identification and reporting of abnormal operation and/or failure of one or more components of the monitoring system (sensor, acquisition unit, management unit, edge unit, switch, router, etc.). This information can be correlated and also provides support for device management and device management in general.
3) **Measurements:** Quantitative information of the measurement acquired by the sensors. This flow can be also used for sending algorithm results.
4) **Snapshot:** Images acquired on the monitored structure.

3.1 Alarm Management

As regards the “Alarm Management”, the messages to be managed from field systems to the platform IoT are:

- Alarm onset • Alarm reset

While those from IoT platform to the Management Unit:

- Alarm acknowledgement
- Alarm closure

Below is a description of the JSON fields for the aforementioned messages. During implementation, FS reserves the right to increase the number of JSON fields by up to 50% and update the description and message type shown below without incurring any additional charges from the Contractor.

Field	Description	Message Type
MsgId	Unique identifier of the sent message that allows the differentiation of all received messages	Alarm onset Alarm acknowledgement Alarm reset Alarm closure

Machine Translated by Google

IDGateway	Unique identifier of the Management Unit or Edge Unit that sends the information, allows you to quickly trace the affected device	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
SenderId	Unique supplier identifier of the	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
Timestamp	Timestamp of the alarm (timestamp when the problem actually occurred). Format: Milliseconds since Unix Epoch	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
Version	Software version	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
SensorAlarm{AlarmId, Value}	Where AlarmId : Unique identifier of the alarm given the asset Value: Value assumed by the measuring device: “1” for opening	Alarm onset
SensorAlarmClosed{AlarmId, Value}	Where AlarmId : Unique identifier of the alarm given the asset Value: Value assumed by the measuring device: generally “0” for return	Alarm reset
SensorAlarmHandled{AlarmId, Value}	Where AlarmId : Unique identifier of the alarm given the asset Value: Value assumed by the measuring device, “2” for acknowledgement	Alarm acknowledgement
SensorAlarmManagedClosed{AlarmId, Value}	Where AlarmId : Unique identifier of the alarm given the asset Value: Value assumed by the measuring device: “3” for manual closing	Alarm closure
CurrentValue	Current value (above the threshold) of the individual alarm. This allows you to provide information on how much has been exceeded and perform subsequent processing.	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
Area	Sector that identifies the type of alarm (LV, TLC, TE, IS)	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
FaultTimestamp	Alarm timestamp (timestamp when the message was sent to IoT). Format: Milliseconds since Unix Epoch	Alarm onset
Problem Intensity	Text field where the intensity description is entered. This field must be mapped in the master data.	Alarm onset Alarm acknowledgement Alarm reset Alarm closure

Machine Translated by Google

Problem Location	A text field describing the location of the received alarm, providing additional information to the Latitude and Longitude fields. This field must be mapped And in the registry.	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
AlarmText	Free text field where you can add additional information if necessary	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
Alarm Severity Level	Identifier of the severity level of the alarm generated	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
SensorList [{IdUAD, IdSensor, SensorType }]	list of sensors that generated the accompanying alarm (minimum 1) IdUAD: Identifier of the Sensor afferent UAD IdSensor: Unique identifier of the sensor that generated the alarm SensorType: Identifier of the type of belonging of the sensor	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
SpanId	SAP identifier of the span of the work affected by the alarm.	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
Pkalarm	Unique alarm identifier. Generated by the field system to identify repeated identical alarms (same pkalarm) from different sequential alarms (different pkalarm).	Alarm onset Alarm acknowledgement Alarm reset Alarm closure

Table 1 Application/ JSON for alarm management

Alarms with the same primary key (Pkalarm) must not be sent back to the IoT Platform unless a realignment operation is explicitly requested. However, they must be sent if they undergo a status change, such as a change from "Alarm Onset" to "Alarm Closure," or if their severity changes.

Additionally, in the event of an alarm affecting multiple spans, the field systems must send an alarm message for each affected span. Each message will have the same pkAlarm and will contain the sensors affected by the alarm (via the SensorList array) that pertain to the specific span (spanId).

3.1.1 Periodic Alarm Realignment

A configurable flow from the field to the IoT platform for periodic alarm status realignment will also be required, with the same characteristics as the flows described above. The details will be defined during the implementation phase.

Machine Translated by Google

The process is based on the use of the MQTTS protocol for transmitting and receiving messages between the Management Unit/Edge and the central IoT platform.

Process Phases:

1. Sending the realignment request: The IoT platform publishes an MQTT message with a JSON payload containing the time interval for which it is requesting the alarm status. The JSON payload has the following structure:
   a. IDGateway: Management Unit/Edge Unit Identifier
   b. startTimestamp: Start timestamp of the period to be analyzed.
   c. endTimestamp: End timestamp of the requested period
   d. ValueStatus: Alarm status. Value (0, 1, 2, 3 as described). Blank if all.
2. Request processing: The Management Unit/Edge Unit receives the request and retrieves information about the alarms stored in the specified period.
3. Sending the response with the alarm status: The Management Unit/Edge Unit publishes MQTT messages with the updated status of the alarms detected in the requested time interval and in the specific topic. The message payload is JSON type and respects the fields described in the previous chapter *(3.1 Alarm Management).*

3.2. Fault Management

As regards **"Fault Management",** the messages to be managed from the Edge to the IoT platform are:

• Fault onset • Fault recovery

While those from IoT platform to the Management Unit:

• Fault acknowledgment
• Fault closure

Below is a description of the fields and their required presence in the message type. During implementation, FS reserves the right to increase the number of JSON fields by up to 50% without the Contractor being able to request additional costs.

Field	Description	Message Type
MsgId	Unique identifier of the message sent that allows us to differentiate all messages	Fault occurrence Fault return Viewing fault received Fault closure
IDGateway	Unique identifier of the Management Unit or Edge unit that sends information, allows you to quickly trace the affected device	Fault occurrence Return fault Fault vision taken Fault closure
SenderId	Unique identifier of the supplier (from map in registry)	Fault occurrence Return fault (Review fault (record/ Fault closure
Timestamp	Timestamp of the fault sending (Milliseconds) since Unix Epoch)	Fault occurrence (Return failure Fault vision taken

Machine Translated by Google

		Fault closure
FaultTimestamp	Failure timestamp (Milliseconds timestamp) Unix Epoch)	Fault onset since
Area	Sector that identifies the type of fault fault (LV, TLC, TE, IS)	Fault occurrence Fault return
FailureText	Free text field where you can additional information if necessary necessary	Fault occurrence Return fault add Fault inspection Fault closure
Failure Severity Level	Failure severity level identifier generated (to be recorded/mapped in the registry)	Fault occurrence Return fault Viewing of the fault Fault closure
Version	Software version	Fault occurrence Return fault Fault vision taken Fault closure
SensorList [{IdUAD, IdSensor, SensorType }]	list of sensors that generated the fault accompanied by lat and lon (minimum 1) IdUAD: Reference UAD IdSensor: Unique identifier of the sensor that generated the fault SensorType: Identifier of the type to which the sensor belongs	Fault occurrence Return fault Fault vision taken Closing fault
SensorFailure { Failure Id, the asset ID generated by the control unit and Value}	Identify the fault sending notification to the IoT Platform Fault FailureId: Unique identifier of the given failure assigned to the specific fault; the same ID will be passed throughout the fault management cycle (to be recorded/mapped in the registry) Value: 1=indicates opening	Onset
SensorFailureHandled {FailureId,Value}	Identify the notification of taking charge of a fault on the IoT platform FailureId: Unique identifier for the failure for the asset. This is the ID generated by the control unit and assigned to the specific failure; the same ID will be passed throughout the failure management cycle (to be recorded/mapped in the master data). Value: type of indentation (value 2 indicates acknowledgement)	

Machine Translated by Google

SensorFailureClosed {FailureId,Value}	Identify the return of a fault from Field System to IoT Platform FailureId: Unique identifier for the failure for the asset. This is the ID generated by the control unit and assigned to the specific failure; the same ID will be passed throughout the failure management cycle (to be recorded/mapped in the master data). Value: indentation type (value 1 indicates indentation)	Return fault
SensorFailureManagedClosed control unit be {FailureId,Value}	Identify the closure in fault management FailureId: Unique identifier of the failure for the given asset. Represents the ID generated by the and assigned to the specific failure; the same id will passed throughout the entire fault management cycle (to be recorded/mapped in the registry) Value: return type (value 3 indicates operator closure)	Fault closure
SpanId	SAP identifier of the span of the work affected by the alarm.	Alarm onset Alarm acknowledgement Alarm reset Alarm closure
Pkfailure	A unique fault identifier. Generated by the field system to identify repeated identical faults (same pkfailure) from different sequential faults (different pkfailure).	Fault onset Fault return Fault acknowledgement Fault closure

Table 2 Application/ JSON for fault management

Failures with the same primary key (Pkfailure) must not be sent back to the IoT Platform unless a realignment operation is explicitly requested. However, they must be transmitted

Machine Translated by Google

in the event that they undergo a change in status, for example in the transition from "Fault Onset" to "Fault Closure", or in severity.

Additionally, in the event of a failure involving multiple spans, the field systems must send a failure message for each affected span. Each message will have the same pkFailure and will contain the sensors affected by the failure (via the SensorList array) that pertain to the specific span (spanId).

3.2.1 Periodic Fault Realignment

A periodic fault status realignment flow (from the field to the IoT platform) must also be provided, configurable with the same characteristics as the flows described above. The details will be defined during the implementation phase.

The process is based on the use of the MQTT protocol for transmitting and receiving messages between the Management Unit/Edge and the central IoT platform.

Process Phases:

1. Sending the realignment request: The IoT platform publishes an MQTT message with a JSON payload containing the time interval for which it is requesting the fault status. The JSON payload has the following structure:
    a. IDGateway: Management Unit/Edge Unit Identifier
    b. startTimestamp: Start timestamp of the period to be analyzed.
    c. endTimestamp: End timestamp of the requested period
    d. ValueStatus: Fault status. Value (0,1,2,3 as described). Blank if all.

2. Request Processing: The Management Unit/Edge Unit receives the request and retrieves the information on faults stored in the indicated period.
3. Sending the response with the fault status: The Management Unit/Edge Unit publishes MQTT messages with the updated status of the faults detected in the requested time interval and in the specific topic. The message payload is JSON type and respects the fields described in the previous chapter *(3.2 Fault Management).*

3.3 Measure Management

For this component, two data sending modes must be provided: PUSH mode and PULL mode.

The primary sending mechanism in PUSH SFTP mode will be secure, for all types of measurements and, optionally, the MQTTs protocol for measurements with small payloads (KB order); above this, the use of the MQTTs protocol is not recommended, as is the case for current dynamic measurements with a size of several GB.

The secondary mechanism (this mechanism comes into play when there is a system downtime to recover measurements not sent during the downtime period) is of the PULL type and consists in the Management Unit / Edge Unit exposing a RESTful HTTPs service that can be queried as needed by the IoT platform to request the data collected in a given time window.

This service will allow the Management Unit/Edge Unit to process the request, and the files will then be sent via SFTP protocol. The process is described in chapter *3.3.1 Pull Mode.*

Machine Translated by Google

Below is the description of the fields present in the JSON transmitted by the Management Unit/Edge to the IoT Platform. During implementation, FS reserves the right to increase the number of JSON fields by up to 50% and update the description and message type shown below, without the Contractor being able to request additional charges.

Field	Description
MsgId	Unique identifier of the sent message that allows us to differentiate all received messages
IDGateway	Unique identifier of the Edge Gateway (Management Unit/Edge Unit) that sends the information, interested Unique
SenderId	identifier of the supplier (to be registered/mapped in the master data)
SpanId	SAP identifier of the bridge span
SensorTimestamp	Timestamp of the measurement detected. Format: Milliseconds since Unix Epoch
SenderFsamp	Sampling rate of the data contained Timestamp of
Timestamp	the message sending measurement
Version	Software version
Period	Message forwarding period (it is 1 in case of sending a variation)
Measures[{IdUAD, IdSensor, Value, Type, UMSensorValue, axis}]	Array of measurement values detected by the measuring device IdUAD: Unique identifier of the UAD on which the sensor is certified IdSensor: Unique sensor identifier UMSensorValue: Unit of measurement of the value sent Value: value of the measurement Axis: code of the measurement axis Type: Description of the physical quantity
MeasureTypeId	ID to identify the final recipients of the measure, not mandatory (to be registered/ mapped in the registry)

Table 3 Application/JSON for measurement management

To transmit data via SFTP, a CSV (inside a ZIP file) with the same fields will need to be produced, where each element of the Measure array will produce one row of the CSV.

The structure of the CSV file may, however, be functional to the measurements sent and will be defined in detail with RFI during the implementation phase.

Submitted files must have the following nomenclature:

- *<assetCode>_<Management unit/ Edge unit>_<yyyy-mm-dd_hh-mm-ss>_<measurement type>_ <transferId>.csv*
- *<assetCode>_<Management unit/ Edge unit>_<yyyy-mm-dd_hh-mm-ss>_<measurement type>_ <transferId>.zip*

Below is a representation of the individual placeholders:

- *assetCode:* SAP identification code of the monitored infrastructure
- *Management unit/ Edge unit:* unique identifier
- *yyyy-mm-dd_hh-mm-ss:* timestamp of file sending
- *measurement_type:* type of measurement present in the file
- *transferId:* unique identifier of the transmission corresponding to the value returned in output by the */dfm/ device/ {IDGateway}* service

The data must be organized into files and directories by type according to the format and method that will be reviewed together with RFI during the implementation phase.

Machine Translated by Google

This file creation will be regulated by a lock file creation logic that will be discussed during the implementation phase with the FS group.

Data transfer control via checksum is also provided.

If a particular supplier's Management Units/Edge Units have data processing algorithms that RFI deems useful to acquire into the IoT platform, it reserves the right to integrate the sending of the results using the same flow by appropriately modifying the payload and transmission method during the implementation phase.

3.3.1 PULL mode

Each Edge Unit/Management Unit must have a RESTful service with an API for retrieving readings, in the event of malfunctions encountered in the primary mode (PUSH) or specific modes requested by the IoT platform, and which takes the type of measurement and the sampling time window as input.

Since the amount of output data cannot be estimated a priori, as the time window could be arbitrarily large, the sending of the requested measurements via the SFTP protocol was planned in any case.

This chapter describes the operating logic and interfaces of the services used to transfer measurement data from the Management Unit and Edge Units to the central IoT platform, based on the device and a time interval specified in input.

The process is as follows:

1. **Start transfer**
   - The central IoT platform initiates the process by calling a GET service (/dfm/device/{IDGateway}) exposed by the Management Unit or the Edge Unit. • In response, the

   Management Unit provides: o A unique
   transfer identifier (transferId).
   o Status of taking charge
2. **Generate CSV files**
   - The Management Unit or the Edge Unit, in asynchronous mode, starts an internal process to retrieve data based on the requested search criteria and with reference to the transferId
3. **File transfer via SFTP**
   - After file generation, the Management Unit/Edge Unit opens an SFTP connection to the central IoT platform to transfer the csv file (contained in a .zip file) to a specific path

   - The generated CSV file must contain the same fields described in chapter *3.3 Measurement Management.*
4. **Verify File Integrity via Checksum**
   - To ensure greater control during the transfer, it is necessary to calculate the file's checksum before sending it. Once the transfer is complete, the checksum must be recalculated on the received file and compared with the initial one. If the values match, the transfer was successful and without alterations, and in this case, the next phase is carried out.

5. **Completion notification**
   - Whenever the Management Unit/Edge Unit sends a measurement file, a "pilot file" must first be created in the same destination directory,

Machine Translated by Google

named like the data file it refers to except for the extension which must be ".lock"

- This file must be empty and will serve as a semaphore to indicate that the related measurement file is still being written and cannot be read. Once writing is complete, the pilot file (.lock) must be removed, to allow the full file (.zip) to be read.

The REST service interface is described below: dfm/device/{IDGateway}.

The REST service exposed by the Edge Unit/Management Unit has the following interface:

- **http-method:** GET

- **url-path: /**dfm/device/{IDGateway}

- IDGateway: represents the identifier of the Management Unit/Edge Unit

The input values for the search time window must be entered within the **query-parameters structure.** Their descriptions are as follows:

	Description
startTimestamp	Timestamp of the start date of the data recovery time window
endTimestamp	Timestamp of the end date of the data recovery time window
type	Type of measurement requested

The output payload is in JSON format and contains the following fields:

Field	Description
transferId	Transfer ID
status	Value indicating whether the file transfer was accepted or not
path	Represents the path where the zip file will be saved on the central IoT platform
fileName	File name including extension
errorMessage	Error description. Valued if the call processing fails.

Below is an example of a correct response (http-status=200) from the Management Unit/Edge Unit:

```
{
  "transferId": "12345",
  "status": "accepted",
   "path": ""{root}/dfmFileName.csv
  "fileName": "FileName.csv",
  "errorMessage ": ""
}
```

Files must follow the naming rules described in chapter 3.3

3.4 Snapshot Management

For this component, two data sending modes must be provided: PUSH mode and PULL mode, similar to that of the measurements.

Machine Translated by Google

The primary mechanism for sending in PUSH SFTP mode will be secure, using the protocol MQTTs for images requiring a small file size, only if encoded in Base64; if the image size is large, sending must be done via SFTP (with zip and image resolution to be defined).

The secondary mechanism (this mechanism comes into play when there is a system downtime to recover images not sent during the downtime) is of the PULL type and consists in the Management Unit / Edge Unit exposing a RESTful HTTPs service that can be queried as needed by the IoT platform to request the images collected in a window.

For **"Snapshots,"** JSON messages sent to the IoT Platform transmit the image, encoded in Base64. They will be sent periodically, at set intervals (once or more a day).
Additionally, the payload structure is designed to receive multiple snapshots in a single message.

Below is a description of the fields in the JSON message in the case of snapshot management:

Field	Description
MsgId	Unique identifier of the sent message that allows the differentiation of all received messages
IDGateway	Unique identifier of the Management Unit/Edge Unit, allows you to quickly trace the affected device
SpanId	SAP Unique Identifier of the span
IdUAD	Unique UAD Identifier
SenderId	Unique supplier identifier
Timestamp	Timestamp of message sending Format: Milliseconds since Unix Epoch
Version	Software version
Period	Message forwarding period
Snapshots	Element that can contain multiple images (minimum 1)
SnapshotId	Unique identifier of the snapshot received
SnapshotValue	Base64 value of the received image
SnapshotTimestamp	Timestamp of the received image
CameraId	Identifier of the camera associated with the Sender

3.4.1 Bulk image sending

For transferring images from the Management Unit/Edge unit to the central platform, in the case of medium/ large sized images, two distinct methods are provided:

1. **Batch Mode**
   - Images are sent in **batch** mode according to a predefined schedule.
   - Sending occurs via **SFTP,** ensuring secure file transfer from the Management Unit/Edge to the IoT platform. Notification of file transfer completion is described in the “Completion Notification” section of the following transfer mode.

   - File Integrity Verification via Checksum: To ensure greater control during the transfer, it is necessary to calculate the file checksum before sending. Once the transfer is complete, the checksum must be recalculated on the received file and compared with the initial one. If the values match, the transfer was successful and without errors.
     alterations and in this case we move on to the next phase.
   - Completion notification:

Machine Translated by Google

or the Management Unit/Edge Unit must first create a "pilot file" in the same destination directory, named the same as the data file to be transferred but with the extension ".lock"

This file must be empty and will serve as a semaphore to indicate that the related measurement file is still being written and cannot be read. Once writing is complete, the pilot file (.lock) must be removed to allow the full file (.zip) to be read.

2. On-demand mode

- The central platform initiates the transfer by calling a **GET** service (dfm/device/snapshot/) exposed by the Management Unit/Edge Unit. This call triggers the image retrieval and sending process.
- Once the process is started, the image transfer occurs asynchronously via **SFTP.** It is important to note that the images must be included in a .zip file before being sent.

- File Integrity Verification via Checksum: To ensure greater control during the transfer, it is necessary to calculate the file checksum before sending. Once the transfer is complete, the checksum must be recalculated on the received file and compared with the initial one. If the values match, the transfer was successful and without errors.
  alterations and in this case we move on to the next phase.
- Completion notification:

  or the Management Unit/Edge Unit must first create a "pilot file" in the same destination directory, named the same as the data file to be transferred but with the extension ".lock"

  This file must be empty and will serve as a semaphore to indicate that the related measurement file is still being written and cannot be read. Once writing is complete, the pilot file (.lock) must be removed to allow the full file (.zip) to be read.

The REST service interface is described below: dfm/device/snapshot/{CameraId}.

The REST service exposed by the Management Unit/Edge Unit that is responsible for initiating image acquisition has the following interface:

- **http-method:** GET

- **url-path:** dfm/device/snapshot/{CameraId}

  - CameraId: represents the unique identifier of the camera

The input values for the search time window must be entered within the **query-parameters structure.** Their descriptions are as follows:

	Description
startTimestamp	Timestamp of the start date of the data recovery time window
endTimestamp	Timestamp of the end date of the data recovery time window

The output payload is in JSON format and contains the following fields:

Field	Description

Machine Translated by Google

transferId	Transfer ID
status	Value indicating whether the file transfer was accepted or not
path	Represents the path where the zip file is saved on the central IoT platform
fileName	File name including extension
errorMessage	Error description. Valued if the call processing fails.

Below is an example of a correct response (http-status=200) from the Management Unit:

```
{
  "transferId": "12345",
  "status": "accepted",
  "path": ""{root}/dfm/snapshot",",
  "fileName": "filename.zip ",
  "errorMessage ": ""
}
```

In both cases (PUSH/PULL) the .zip file, transferred from the Management Unit/Edge Unit to the IoT platform, must respect the following structure for each image sent:

- A metadata file, containing the following fields: *MsgId, IDGateway, SenderId, Timestamp, Version, SnapshotId, SnapshotTimestamp, NameFile, CameraId.* For a description, please refer to chapter 3.4.

- File containing the acquired image.

The file names are as follows:

- Captured image file: *<assetCode>_<Management/ Edge unit>_<yyyy-mm-dd_hh-mm-ss>_ <CameraId>.<format>*
- File of the metadata: *<assetCode>_<Managementunit/ Edgeunit>_<yyyy-mm-dd_hh-mm-ss>_<transferId>_metadata.csv*
- Zip file: *<assetCode>_<Management Unit/ Edge Unit>_<yyyy-mm-dd_hh-mm-ss>_ <CameraId><transferId>.zip*

3.5 Topics to Use Below are

examples of the 5 MQTT topics to which messages are expected to be received. The corresponding legend is provided:

CENTRAL/TR0001-XX-XXXX-XXX-XXX/ALARM/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX/ALARM/DEVICE/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX/MEASURES/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX/FAILURE/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX/FAILURE/DEVICE/<IDGateway>

CENTRAL /TR0001-XX-XXXX-XXX-XXX/SNAPSHOT/<IDGateway>

In which:

Machine Translated by Google

- CENTRAL: reference compartment code. (It may be RM in the case of DOIT Rome or BA in the case of DOIT Bari and so on).
- TR0001-XX-XXXX-XXX-XXX: INRETE2000 code of reference bridge, technical headquarters of fifth level.
- ALARM/<IDGateway>: characterizes the topic to which the onset and return are sent of alarms from field systems to the IoT platform;
- <GatewayID>: gateway ID, i.e. Edge Unit/Management Unit chosen as an example, replaceable and to be agreed upon. It uniquely identifies the device integrated with the IoT Platform. This topic is used for communication from the Management Unit to the IoT platform.

- ALARM/DEVICE/<IDGateway>: characterizes the topic to which alarm acknowledgement and closure messages are sent from the IoT platform to the field systems. •

MEASURES/<IDGateway>: characterizes the topic to which measurements are sent. •

FAILURE/<IDGateway>: characterizes the topic to which failures are sent. •

SNAPSHOT/<IDGateway>: characterizes the topic to which snapshots are sent.

4. Device Management

The Device Management component offered by the IoT Platform plays a fundamental role in managing field devices within the monitoring system. Thanks to its features, it allows you to configure and remotely control devices (sensors, UADs, Management Units, and Edge Units) in based on operational needs, monitor their status through advanced diagnostic tools and manage their administration efficiently.

4.1 Device Registry Management

In order to provide Device Management functions, the IoT Platform must manage the Device Registry, through which it is possible to register, update and manage sensors, UADs (Data Acquisition Units), and Management and Edge Units, as well as having the coordinates to correlate IoT data with the global identification codes of the assets (for example, SAP codes for bridges).

Global identifiers are always defined centrally and must be communicated to vendors to correctly map local IoT devices.
With this feature, users can maintain an up-to-date inventory of devices, monitor their evolution over time, and ensure efficient management of the entire IoT ecosystem.

For a device to send data to the IoT Platform, it must first be registered in the Access Control List. If a Management Unit, Edge Unit, or other entity that interfaces with the IoT platform is not registered, the platform will not accept the transmitted data, thus ensuring strict control over the integrity and security of the information managed by the system.

Device registration in the Device Master Data module will be managed through an offline process, following testing of the sensor installation. The content and methods will be agreed upon with RFI.

Each Management Unit and Edge Unit must have associated network endpoints for the HTTPS protocol (HTTPS Endpoint) needed to address it globally. These endpoints must be recorded in the registry.

Machine Translated by Google

It will also be necessary to track the geographic location of all the elements that make up the field network in the master data. In any case, all the attributes to be recorded in the master data will be defined in detail during the implementation phase.

When the sensor is turned on, the relevant Management Units will send a LOGIN message with the relevant details defined in the next paragraph.

4.2 Command Management

The main management operations you can perform include:

- Login: Sensor Registration/ UAD/Management Unit/Edge Unit
- Restart: Restart sensors/ UAD/Management Units/Edge Units
- Stop: Stop sensors/ UAD/Management Units/Edge Units • Send configurations regarding data logging periods, trigger thresholds and cutoffs, pre-trigger and post-trigger window durations and sampling rates
- Request current monitoring configuration and request diagnostic/lifetime status report of devices (e.g. Sensors, UAD, Management Unit, Edge Unit)

- Time synchronization, for example, NTP (Network Time Protocol) • Firmware update • Sensor group inhibition to ignore alarms and detected data • Sensor group reset to receive alarms and detected data again

FS reserves the right to modify and/or increase the above-mentioned operations by 50% without requiring the Contractor to incur additional costs. If the IoT systems provided have additional device management features, these must be made available for integration into the Device Management system to better organize remote control processes.

To ensure the implementation of these features, the Management unit and the Edge unit must be equipped with a software module that implements the MQTT Publisher and Subscriber model, developed by the vendor, capable of:

- correctly interpret commands given remotely via MQTTS (Command request)
- send a confirmation of receipt via an MQTTS ACK (Command response) message, specifically enhancing the JSON field of the response: *Response {DeviceId, Success, Value}*
- perform the required actions based on the command received.
- notify the new status of the device with an MQTTS message, appropriately enhancing the JSON field of the response: SensorsStatus *{DeviceId, Value}*

The payload of the REQUEST and RESPONSE messages is in JSON format and follows the structures shown. right away.

REQUEST Message

Field	Description
MsgId	unique identifier of the sent message
IDGateway	Unique device identifier. Can be filled with Management Unit ID or Edge Unit ID.

Machine Translated by Google

Timestamp	Alarm sending timestamp
Version	Software version
Period	Message forwarding period
RequestId	Unique identifier of the command request
Command [{DeviceId, Value, Params[key, value]}]	Array object containing the command: • DeviceId: Identifier of the sensor or UAD to which the command is sent. It can be set to NONE when sending commands to Management Units or Edge Units. • Value: Value of the command sent to be associated with the sensor/UAD/ Management Unit/Edge Unit • Params[key, value]: key value array for setting variable parameters depending on the command o Key= parameter type o Value = parameter value. It can also be the URL of the repository from which to download the firmware to install. The domain of this array will be defined during the implementation phase.

Below is the description and domain of the Value field values in the REQUEST message:

- LOGIN: Sensor/UAD/Edge Unit/Management Unit registration
- RESTART: restart arranged
- STOP: device stops
- SETUP: change monitoring parameters
- STATUS: Request current monitoring configuration. This class includes monitoring the operating status of the sensor/UAD/Management Unit/Edge Unit (e.g. battery level, or power on/off).
- SYNC_NTP: NTP time synchronization
- FIRMWARE: Remotely update the device firmware
- EXCLUDE: Inhibit a group of sensors to ignore alarms and detected data
- RESTORE: Reset sensor group to receive alarms and detected data again

During the implementation phase, FS reserves the right to increase the number of JSON fields by up to 50% and to review the logic for evaluating these fields without the Contractor being able to request additional costs.

Below is the description of the RESPONSE message:

Field	Description	Message Type
MsgId	unique identifier of the sent message ACK message	Status Message
IDGateway	Unique device identifier. It can be valued with the Unit ID Edge Unit Management or ID	ACK message Status Message
Timestamp	Alarm sending timestamp	ACK message Status Message
Version	Software version	ACK message Status Message
Period	Message forwarding period	ACK message Status Message
RequestId	Unique identifier of the command request (see previous legend)	ACK message Status Message

Machine Translated by Google

Response [{DeviceId, Success, Value}]	Array of response container objects • DeviceId: Identifier of the sensor or UAD towards which the action is taken command. It can be set to NONE when sending commands to Management Units or Edge Units • Success: Value Boolean (true/false) indicating the success or failure of the command. • Value: Value attributed to the sensor following the execution of the command or identification value of the command	ACK message
SensorsStatus [{DeviceId, Value}]	Array of objects containing the device state following the state change: • DeviceId: Sensor or UAD identifier • Value: Sensor/ UAD value taken over by The object is optional and is filled when the device sends its new updated state	Status Message

During the implementation phase, FS reserves the right to increase the number of JSON fields by up to 50% and to review the logic for evaluating these fields without the Contractor being able to request additional costs.

The command sequence must also be configurable

The states that are described in this chapter are all sent in response to request and response commands, with the exception of the RESTART/STOP state which must also be sent with a configurable periodicity of x minutes (default 10 minutes).

4.3 Topic to use CENTRAL/

TR0001-XX-XXXX-XXX-XXX /DM/CMD/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX /DM/CMD/FEEDBACK/<IDGateway>

CENTRAL/TR0001-XX-XXXX-XXX-XXX /DM/STATUS/<IDGateway>

Legend

- CENTRAL: reference compartment code. (It can be RM in the case of DOIT of Rome or BA in the case of DOIT of Bari and so on). • TR0001-XX-XXXX-XXX-XXX SAP code of the Bridge
- DM/CMD topic for sending the Device Management command from the IoT Platform to the field characterizes the topic to which the commands are sent.
- DM/CMD/FEEDBACK topic for response from Field to IoT Platform to notify the command ACK.
- DM/CMD/STATUS topic for sending device status

Machine Translated by Google

5. Remote access to field equipment by RFI

RFI users are expected to be able to access supplier data from the field; each supplier will be required to provide the access solution to their own software platforms, residing on the field units (UADs, management PCs, edge PCs, etc.) during the implementation and operation phase of their field solution. Specifications and user manuals will be required for RFI users to operate the systems.

To ensure the security of corporate networks, firewalls will be installed to segregate IT and OT networks, thus protecting corporate systems from field operations infrastructure.

6. Remote access to field devices by Suppliers

Third-party suppliers will be able to access the field systems through an IPsec VPN connection specifically configured for access to the RFI network, the configuration parameters of which will be shared during the execution phase of the contract.

The protocols allowed within the VPN will be exclusively encrypted (e.g. https, SSH, SFTP, etc.). Clear protocols will not be authorized. The contractor's solution must, in any case, be represented in the design documents by the contractor and submitted to RFI for approval.

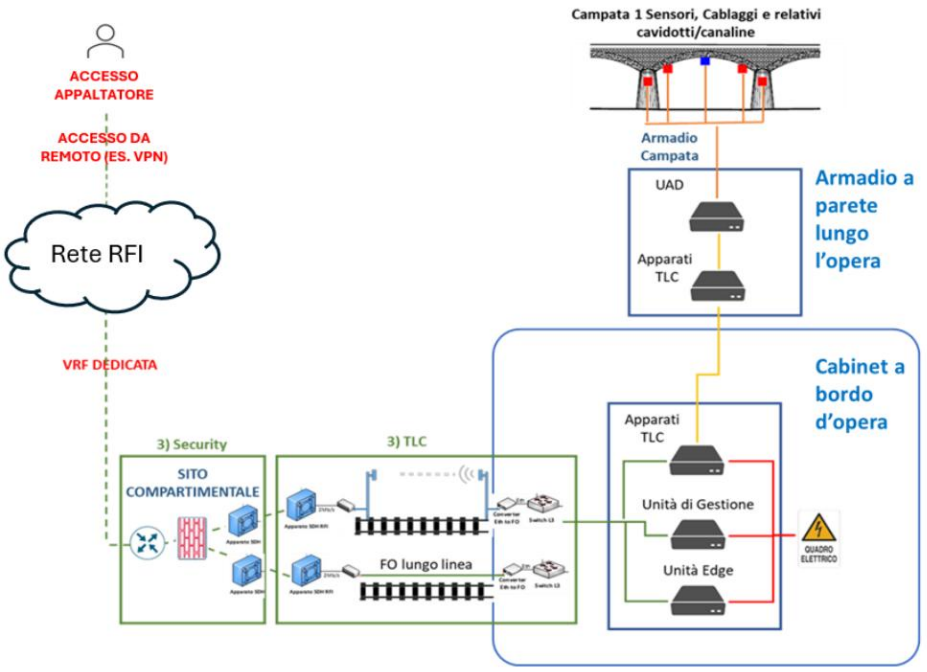

Figure 3 – Remote access to field devices by Suppliers via optical fibre and RFI fixed network

Machine Translated by Google

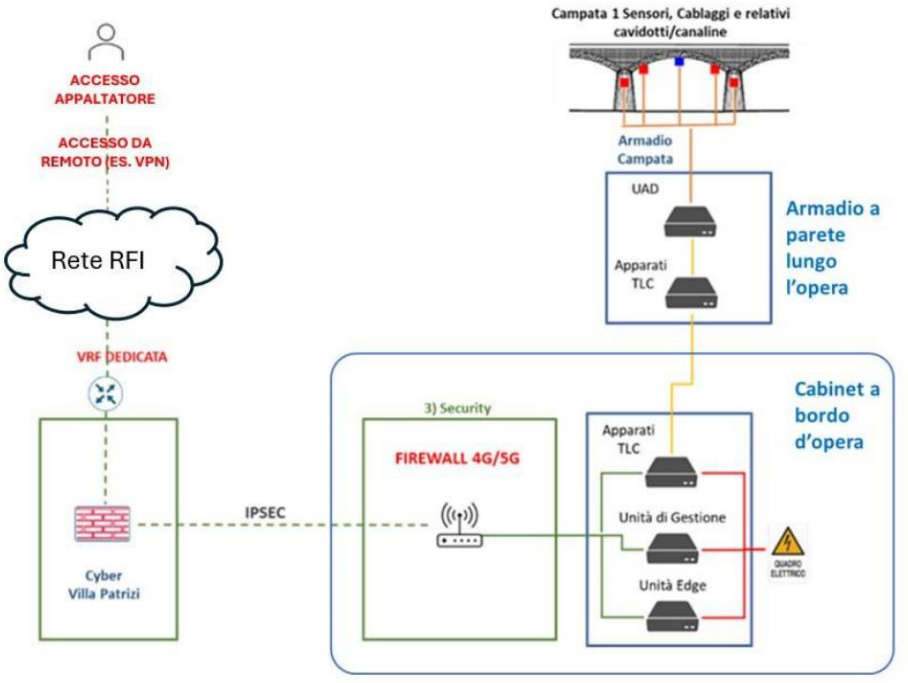

Figure 4 – Remote access to field devices by Suppliers via IPSEC VPN over the Internet